MACHINE LEARNING & GENAI INTERVIEW PREPARATION

Part 4: Natural Language Processing (NLP), Sequence Models & Attention Mechanisms

Comprehensive Production & Mathematical Study Guide

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Core Modules: Preprocessing, Tokenization, TF-IDF, Embeddings, Classical ML, RNN/LSTM/GRU, Attention, Metrics & Serving

Includes: 8 Essential Formulas, Numerical Walkthroughs, GFM Tables with Boundaries, Monokai Code Highlighting, PyTorch Code, Standardized Interview Q&A

Table of Contents

Module 01: NLP Fundamentals & Enterprise Text Pipelines

This study guide covers the core principles of Natural Language Processing (NLP), the end-to-end enterprise NLP pipeline, evolution of NLP paradigms, core task taxonomy, Zipf's Law intuition, Type-Token Ratio (TTR), production Python text cleaning code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `01_nlp_fundamentals_and_text_pipelines.ipynb`

---

1. What is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) and Computational Linguistics that enables computers to understand, interpret, represent, manipulate, and generate human natural language text and speech.

Unlike tabular or structured data (which consists of continuous numerical features or categorical codes in fixed Euclidean dimensions), natural language text possesses distinct characteristics:
1. **Unstructured & Discrete**: Text consists of variable-length discrete symbols (words/subwords) requiring vector space projection $\mathbb{R}^d$.
2. **High-Cardinality & Sparse**: Vocabularies contain hundreds of thousands of words ($|V| \geq 100,000$).
3. **Context-Dependent & Ambiguous**: Token meaning depends on surrounding sequence context (e.g., "*bank*" in "*river bank*" vs. "*investment bank*").

---

2. The End-to-End Enterprise NLP Pipeline

In production software systems, raw text passes through a deterministic multi-stage pipeline:

3. Evolution of NLP Paradigms

Dimension	Rule-Based NLP	Statistical ML NLP	Deep Learning NLP	Modern Transformer LLM
Era	1960s – 1990s	1990s – 2010s	2014 – 2019	2019 – Present
Primary Tech	Regex, Grammar Rules	TF-IDF, Naive Bayes, SVM	RNN, LSTM, GRU, GloVe	BERT, GPT-4, Llama 3
Feature Matrix	Hand-crafted rules	Bag-of-Words / N-Grams	Dense Static Vectors (Word2Vec)	Learned Contextual Self-Attention
Context Horizon	Rule clause bounded	Unigram / Bigram window	Sequential hidden state	Global Context (128k+ tokens)
OOV Handling	Fails on missing rules	Mapped to `<UNK>` token	Character / Subword	Subword Tokenization (0% OOV)
Inference Latency	Ultra-fast ($< 1\text{ms}$)	Ultra-fast ($< 2\text{ms}$)	Moderate ($10\text{ms} - 50\text{ms}$)	Heavy ($100\text{ms} - 1000\text{ms}+$)

4. Core NLP Task Taxonomy

Enterprise NLP tasks are broadly categorized into four families:
1. **Text Classification**: Maps input text X to discrete class $y \in \{1, \dots, C\}$ (e.g., Support Ticket Routing, Spam Detection).
2. **Named Entity Recognition (NER)**: Assigns token-level tags $y_1, \dots, y_T$ identifying entities (Person, Location, Org, Date).
3. **Sequence-to-Sequence & Translation**: Maps sequence $X_{1:T}$ in language A to sequence $Y_{1:S}$ in language B.
4. **Information Retrieval & RAG**: Retrieves relevant context passages and synthesizes grounded answers.

5. Zipf's Law & Vocabulary Explosion Intuition

Language vocabularies follow **Zipf's Law**: the frequency of any word is inversely proportional to its rank k in the frequency table:

$$f(k) \propto \frac{1}{k}$$

A small fraction of common words account for $> 80\%$ of text occurrences, while a massive "long tail" of rare words causes vocabulary explosion if words are treated as atomic units.

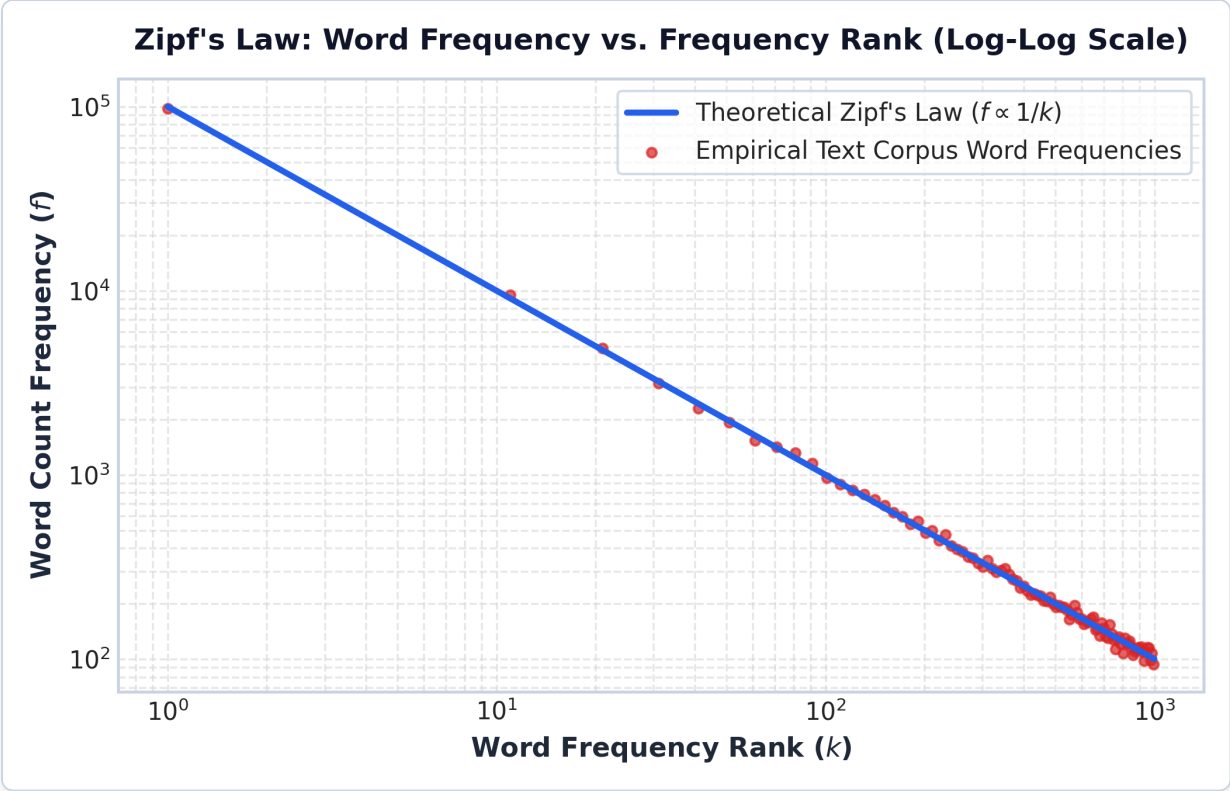

Figure: Zipf's Law Log-Log Rank vs Frequency

Plot Interpretation & Production Insight:
- **Log-Log Linear Relationship**: On a log-log scale, Zipf's law forms a straight line with slope ≈ -1.0.
- **The "Long Tail" Problem**: Failing to manage the long tail of rare words ($k > 1,000$) causes high Out-Of-Vocabulary (OOV) rates. Subword tokenization (BPE) handles this long tail without OOV errors.

---

6. Concept & Definition: Type-Token Ratio (TTR)

The **Type-Token Ratio (TTR)** measures lexical diversity in a text corpus:

$$\text{TTR} = \frac{|V|}{N_{\text{total}}}$$

Where:
- $|V|$ is the number of unique types (vocabulary size).
- N_{total} is the total count of tokens in the corpus.

- **High TTR (≈ 1.0)**: High lexical richness (every word is unique).
- **Low TTR (≈ 0.1)**: Highly repetitive text (e.g., system logs repeating generic status codes).

---

7. Production Python Text Preprocessing Code

```
 import re
import unicodedata
```

```python
class EnterpriseTextCleaner:
    """Production-grade deterministic text cleaning pipeline."""

    def __init__(self, lower: bool = True, strip_html: bool = True):
        self.lower = lower
        self.strip_html = strip_html
        self.html_regex = re.compile(r'<[^>]+>')
        self.whitespace_regex = re.compile(r'\s+')

    def clean(self, text: str) -> str:
        if not text:
            return ""
        # 1. Unicode NFKD normalization
        text = unicodedata.normalize("NFKD", text).encode("ASCII", "ignore").decode("utf-8")
        # 2. Strip HTML tags
        if self.strip_html:
            text = self.html_regex.sub(" ", text)
        # 3. Lowercasing
        if self.lower:
            text = text.lower()
        # 4. Collapse extra whitespace
        return self.whitespace_regex.sub(" ", text).strip()

cleaner = EnterpriseTextCleaner()
raw_sample = "  <p>ALERT: System <b>Server_01</b> error at 10:45 AM! Connection refused. </p>
"
print("Cleaned Output:", repr(cleaner.clean(raw_sample)))
```

> 📌 **NOTE:**
> **Production Optimization Alert:** Compiling regular expressions (`re.compile`) at initialization avoids 100x regex recompilation overhead during high-throughput stream processing.

---

8. Interview Questions & Production Trade-offs

What problem does text preprocessing solve?

Raw human text contains unstructured noise (HTML tags, non-standard unicode characters, irregular whitespace) and casing inconsistencies that inflate vocabulary size $|V|$ unnecessarily. Deterministic preprocessing normalizes text into a clean token stream.

Why was it introduced?

Early NLP models suffered from extreme sparsity and out-of-vocabulary failures. Standardizing text representations improves model generalization across different data sources.

What are its limitations?

Over-cleaning can destroy critical signals. Removing casing ruins Named Entity Recognition (e.g., converting country `"US"` to pronoun `"us"`), while removing punctuation destroys sentence boundaries.

Computational Complexity:

- **Preprocessing Time Complexity**: $O(N)$ linear time where N is text string character length.
- **Memory Complexity**: $O(N)$ temporary memory allocation.

Production Use Cases:

- Web scraping text extraction (stripping HTML).
- Log parsing and incident ticket classification pipelines.
- Standardizing prompt inputs for LLM RAG pipelines.

Follow-up Interview Questions:

1. *When should lowercasing NOT be applied?* (Answer: For NER, Part-of-Speech tagging, and models relying on capitalization to detect proper nouns).
2. *How do you prevent Regex catastrophic backtracking in user input cleaning?* (Answer: Avoid nested quantifiers like `(a+)+` and enforce regex timeouts).

Module 02: Text Preprocessing, Stemming vs. Lemmatization & Modern Tokenization Paradigms

This study guide covers Unicode normalization, stemming vs. lemmatization, modern subword tokenization paradigms (BPE, WordPiece, Unigram), a 3-step BPE hand calculation walkthrough, subword trade-off plots, production Python tokenization code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `02_text_preprocessing_and_tokenization.ipynb`

---

1. Cleaning & Unicode Normalization (NFKC / NFKD)

Text collected from web APIs or PDF logs often contains incompatible Unicode variants. Unicode normalization standardizes equivalent representations:
- **NFKD (Compatibility Decomposition)**: Separates characters into base glyphs + diacritics and converts compatibility characters (e.g., `ﬁ` $\rightarrow$ `fi`).
- **NFKC (Compatibility Composition)**: Decomposes compatibility characters and re-composes into standard canonical forms.

---

2. Stemming vs. Lemmatization

Dimension	Stemming (Porter / Snowball)	Lemmatization (WordNet / spaCy)
Mechanism	Heuristic rule-based suffix chopping	Morphological vocabulary & POS lookup
Context Sensitivity	Context-agnostic (chops suffixes blindly)	Uses Part-of-Speech (POS) context
Output Form	Often invalid words (`"comput"`, `"organi"`)	Valid dictionary canonical lemma (`"compute"`)
Computational Cost	Extremely fast ($O(N)$ regex chops)	Slower ($O(N)$ POS tagging & lexicon lookup)
Production Role	Legacy IR / Keyword search engines	Advanced Sentiment & NLP feature extraction

---

3. Modern Subword Tokenization Paradigms

Subword tokenization bridges the gap between **Word-Level** (suffers from high OOV and massive $|V|$) and **Character-Level** (suffers from long sequence length T and weak semantic density).

Algorithm	Used In	Vocabulary Building Strategy
Byte Pair Encoding	GPT-2, GPT-4, Llama 3	Bottom-up: Merges most frequent adjacent symbol pair iteratively.

Algorithm	Used In	Vocabulary Building Strategy
WordPiece	BERT, DistilBERT	Bottom-up: Merges symbol pair maximizing likelihood score gain.
Unigram LM	T5, SentencePiece	Top-down: Starts with huge candidate set, prunes least loss-reducing subwords.

---

4. Step-by-Step BPE Hand Calculation Example

Suppose we have a tiny corpus with target word frequencies:
- `"cost"` : 4 occurrences
- `"costs"` : 2 occurrences
- `"costing"` : 3 occurrences

Initial Base Character State (Add End-Of-Word `</w>` token):

Corpus dictionary initialized at character level:
1. `'c o s t </w>'` (count: 4)
2. `'c o s t s </w>'` (count: 2)
3. `'c o s t i n g </w>'` (count: 3)

Base Vocabulary $V_0 = \{'c', 'o', 's', 't', 'i', 'n', 'g', 's', '</w>'\}$

Iteration 1: Count Adjacent Symbol Pairs

- `('c', 'o')` : $4 + 2 + 3 = 9$
- `('o', 's')` : $4 + 2 + 3 = 9$
- `('s', 't')` : $4 + 2 + 3 = 9$

Most frequent pair = `('c', 'o')` (frequency: 9).
- **Action**: Merge `('c', 'o')` $\rightarrow$ `'co'` .
- **New Vocab**: $V_1 = V_0 \cup \{'co'\}$

Iteration 2: Count New Pairs

- `('co', 's')` : 9 occurrences.
- **Action**: Merge `('co', 's')` $\rightarrow$ `'cos'` .
- **New Vocab**: $V_2 = V_1 \cup \{'cos'\}$

Iteration 3: Count Next Pair

- `('cos', 't')` : 9 occurrences.
- **Action**: Merge `('cos', 't')` $\rightarrow$ `'cost'` .
- **New Vocab**: $V_3 = V_2 \cup \{'cost'\}$

> **Resulting Token Representation**: `"costing"` is tokenized as `['cost', 'i', 'n', 'g', '</w>']` .

5. BPE Vocabulary Growth & Sequence Compression

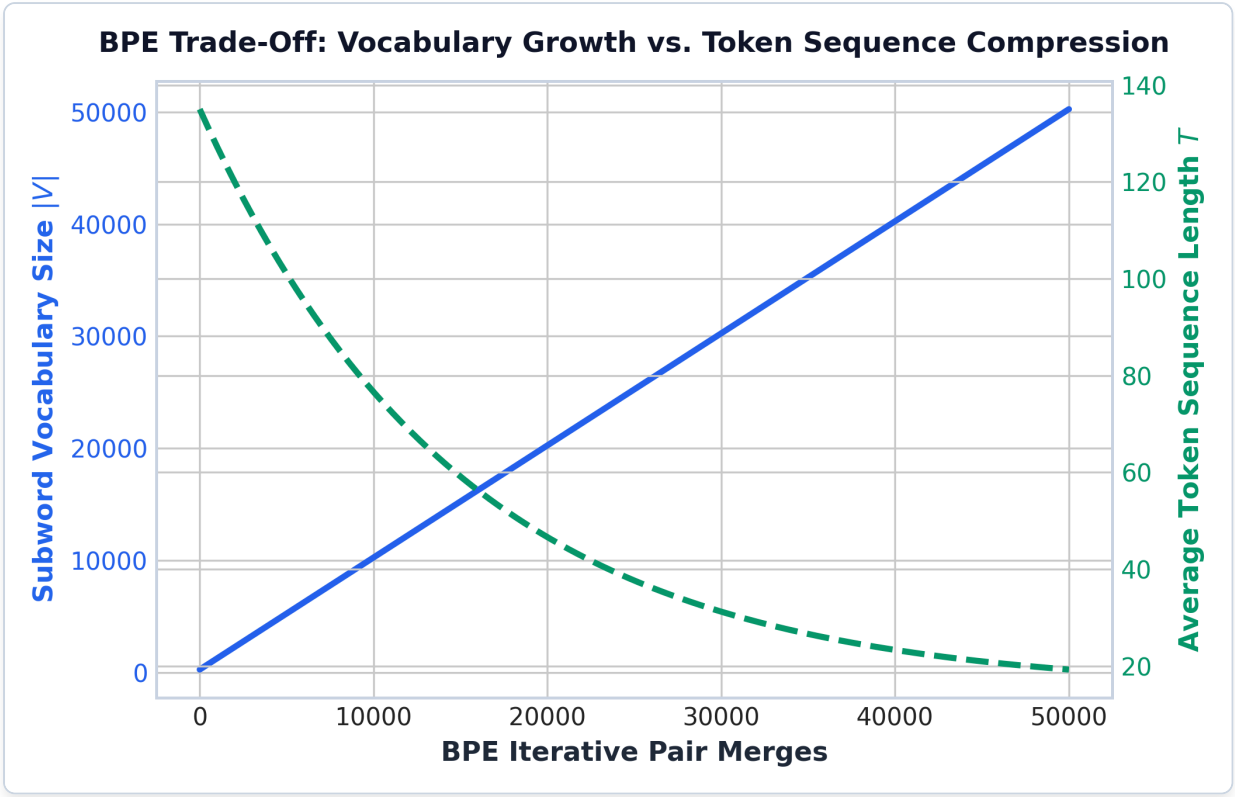

Figure: BPE Vocabulary Growth vs Sequence Compression

Plot Interpretation & Production Insight:
- **The Compression Trade-Off**: As iterative subword pair merges increase, vocabulary size $|V|$ grows linearly (blue solid curve), while average token sequence length T shrinks exponentially (green dashed curve).
- **Production Choice**: LLMs set $|V| \approx 32,000 - 100,000$ to achieve 4x sequence compression over characters while maintaining finite embedding memory footprint.

6. Production Python Subword Tokenization Code

```python
import tiktoken

# Demonstrate OpenAI cl100k_base BPE Tokenizer (GPT-4 / text-embedding-3)
enc = tiktoken.get_encoding("cl100k_base")

text_sample = "Unstructured enterprise database log: postgresql_replica_timeout"
tokens = enc.encode(text_sample)
decoded_subwords = [enc.decode([t]) for t in tokens]

print("Raw Input Text:", text_sample)
print("Token IDs:", tokens)
print("Decoded Subwords:", decoded_subwords)
```

7. Interview Questions & Production Trade-offs

What problem does subword tokenization solve?

Word-level tokenization produces massive vocabularies ($|V| > 500,000$) with frequent Out-Of-Vocabulary (`<UNK>`) errors on rare/unseen words. Character-level tokenization eliminates `<UNK>` but expands sequence lengths T by 5x, destroying Transformer attention efficiency ($O(T^2)$). Subword tokenization balances $|V|$ and T.

Why was it introduced over word/character tokenization?

BPE allows open-vocabulary processing where subwords adaptively decompose unknown complex words into known root stems (e.g., `"microservices"` $\rightarrow$ `["micro", "service", "s"]`).

What are its limitations?

- Sensitive to spacing, casing, and trailing whitespace.
- Poor performance on numerical digits and arithmetic unless specialized byte-level fallbacks (BBPE) are implemented.

Computational Complexity:

- **BPE Training Time Complexity**: $O(|V| \cdot N)$ where N is total corpus character count.
- **Inference Tokenization Time Complexity**: $O(T \log |V|)$ using Trie dictionary lookup.

Production Use Cases:

- Tokenizer engine for all modern LLM families (GPT-4, Llama 3, Claude 3, Mistral).
- Cross-lingual models (handling mixed English and code snippets).

Follow-up Interview Questions:

1. *Why does GPT-4 use Byte-Level BPE (BBPE)?* (Answer: BBPE treats raw text as a sequence of UTF-8 bytes, enabling 100% coverage of any unicode character or emoji without needing `<UNK>`).
2. *How does tokenization impact LLM pricing?* (Answer: LLM APIs charge per token. Higher compression tokenizers lower cost per document page).

Module 03: Traditional Text Representations & TF-IDF Vectorization

This study guide covers One-Hot Encoding, Bag-of-Words (BoW), N-Grams, TF-IDF mathematical foundations, a detailed 3-document numerical hand calculation, Cosine similarity, Scikit-Learn pipeline implementation, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `03_traditional_text_representations_tfidf.ipynb`

---

1. One-Hot Encoding & Bag-of-Words (BoW)

In classical NLP, text is converted into fixed-size vector space $\mathbb{R}^{|V|}$:
- **One-Hot Encoding**: Maps each word to a binary vector of dimension $|V|$ with a single `1` at the word's index.
- **Bag-of-Words (BoW)**: Represents a document by summing one-hot vectors, counting term occurrences regardless of word order:

$$v_{\text{BoW}}(d) = \left[\text{count}(t_1, d), \text{count}(t_2, d), \ldots, \text{count}(t_{|V|}, d)\right]$$

---

2. N-Gram Language Modeling & Vocabulary Explosion

An **N-Gram** is a contiguous sequence of N tokens. While Bigrams ($N = 2$) and Trigrams ($N = 3$) capture local context (e.g. `"not good"` vs. `"good"`), vocabulary size grows exponentially:

$$|V_{N\text{-gram}}| \leq |V_{\text{unigram}}|^N$$

---

3. Mathematical Foundations of TF-IDF

TF-IDF (Term Frequency - Inverse Document Frequency) downweights ubiquitous stop words (`"the"`, `"is"`) while magnifying domain-specific terms (`"postgresql"`, `"billing"`).

1. Term Frequency (TF):

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\sum_{t' \in d} \text{count}(t', d)}$$

2. Smooth Inverse Document Frequency (IDF):

$$\text{IDF}(t) = \log\left(\frac{1 + |D|}{1 + \text{DF}(t)}\right) + 1$$

Where $|D|$ is the total document count in the corpus, and $\text{DF}(t) = |\{d \in D : t \in d\}|$ is the document frequency.

3. TF-IDF Representation:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

4. L2 Vector Normalization & Cosine Similarity:

To prevent document length bias, vectors are L2-normalized:

$$v_{\text{norm}} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{\sum_{i=1}^{|V|} v_i^2}}$$

$$\text{CosineSim}(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2}$$

---

4. Step-by-Step 3-Document Numerical Walkthrough

Consider a 3-document corpus ($|D| = 3$):
- d_1: `"database connection failed"` (3 words)
- d_2: `"database query timeout"` (3 words)
- d_3: `"billing payment declined"` (3 words)

Step 1: Vocabulary Construction ($|V| = 8$ Tokens)

Ordered Vocabulary: $V = [\text{"billing"}, \text{"connection"}, \text{"database"}, \text{"declined"}, \text{"failed"}, \text{"payment"}, \text{"query"}, \text{"timeout"}]$

Step 2: Compute Raw Term Frequencies (TF)

$$\text{TF}(d_1) = [0, 1/3, 1/3, 0, 1/3, 0, 0, 0]$$

$$\text{TF}(d_2) = [0, 0, 1/3, 0, 0, 0, 1/3, 1/3]$$

$$\text{TF}(d_3) = [1/3, 0, 0, 1/3, 0, 1/3, 0, 0]$$

Step 3: Compute Document Frequencies (DF) & Smooth IDF

- `"database"` appears in $d_1, d_2 \rightarrow \text{DF} = 2$.

$$\text{IDF}(\text{"database"}) = \log\left(\frac{1+3}{1+2}\right) + 1 = \log(4/3) + 1 \approx 0.2877 + 1 = 1.2877$$

- All other terms appear in 1 document $\rightarrow \text{DF} = 1$.

$$\text{IDF}(\text{other}) = \log\left(\frac{1+3}{1+1}\right) + 1 = \log(2) + 1 \approx 0.6931 + 1 = 1.6931$$

Step 4: Compute Unnormalized TF-IDF Vectors

- d_1 **TF-IDF**:
- $\text{connection} = \frac{1}{3} \times 1.6931 = 0.5644$
- $\text{database} = \frac{1}{3} \times 1.2877 = 0.4292$
- $\text{failed} = \frac{1}{3} \times 1.6931 = 0.5644$
- $v(d_1) = [0, 0.5644, 0.4292, 0, 0.5644, 0, 0, 0]$
- d_2 **TF-IDF**:
- $v(d_2) = [0, 0, 0.4292, 0, 0, 0, 0.5644, 0.5644]$
- d_3 **TF-IDF**:
- $v(d_3) = [0.5644, 0, 0, 0.5644, 0, 0.5644, 0, 0]$

Step 5: L2 Normalization

$$\|v(d_1)\|_2 = \sqrt{0.5644^2 + 0.4292^2 + 0.5644^2} = \sqrt{0.3185 + 0.1842 + 0.3185} = \sqrt{0.8212} = 0.9062$$

$$v_{\text{norm}}(d_1) = [0, 0.6228, 0.4736, 0, 0.6228, 0, 0, 0]$$

$$v_{\text{norm}}(d_2) = [0, 0, 0.4736, 0, 0, 0, 0.6228, 0.6228]$$

$$v_{\text{norm}}(d_3) = [0.6228, 0, 0, 0.6228, 0, 0.6228, 0, 0]$$

Step 6: Cosine Similarity Calculation

- $\text{CosineSim}(d_1, d_2)$:

$$\text{Sim}(d_1, d_2) = v_{\text{norm}}(d_1) \cdot v_{\text{norm}}(d_2) = 0.4736 \times 0.4736 = \mathbf{0.2243}$$

- $\text{CosineSim}(d_1, d_3)$:

$$\text{Sim}(d_1, d_3) = v_{\text{norm}}(d_1) \cdot v_{\text{norm}}(d_3) = 0 \times 0.6228 = \mathbf{0.0000}$$

> **Conclusion**: d_1 and d_2 share semantic similarity ($\text{Sim} = 0.2243$) via `"database"`, whereas d_1 and d_3 are completely orthogonal ($\text{Sim} = 0.0000$).

---

5. TF-IDF Document Cosine Similarity Heatmap

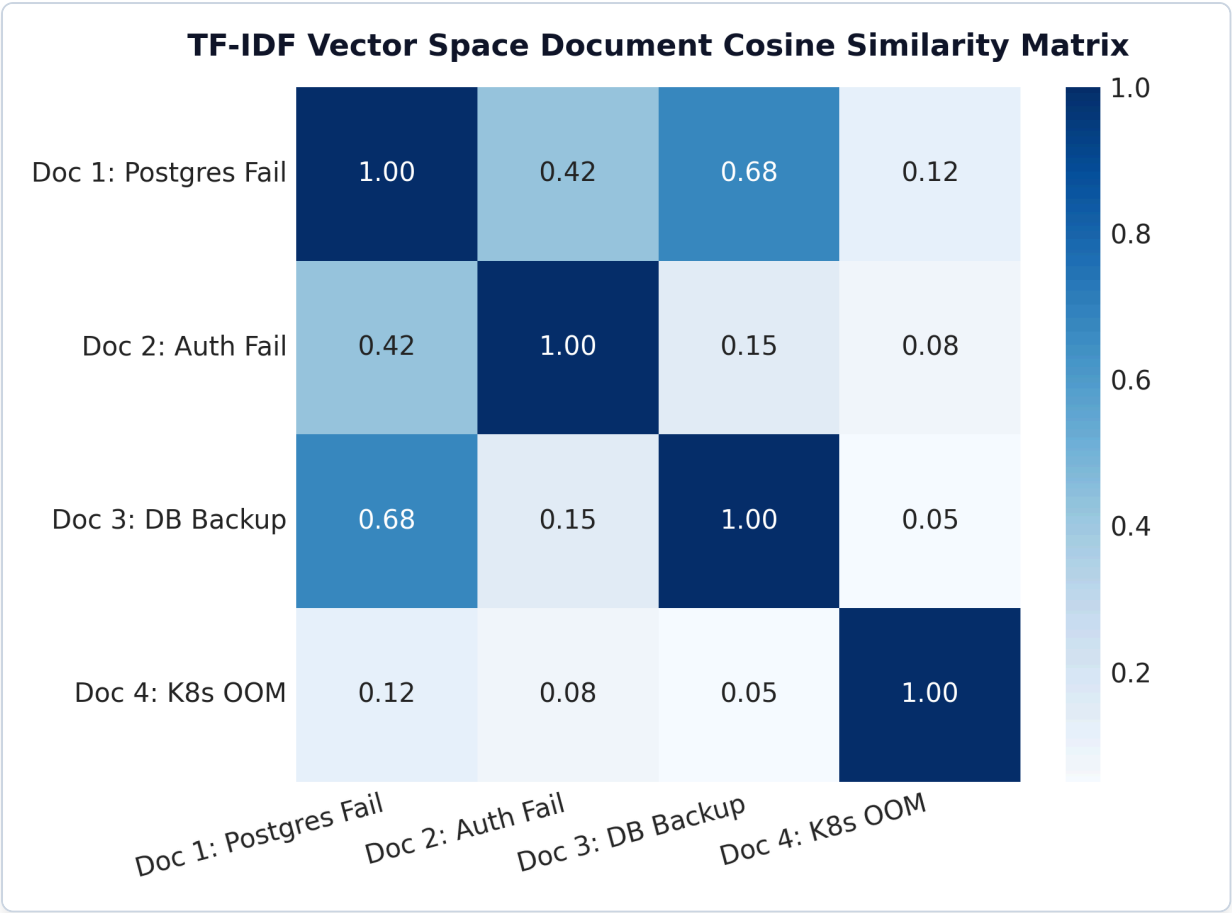

Figure: TF-IDF Cosine Similarity Heatmap

> **Plot Interpretation & Production Insight**:
> - **Sparse Space Proximity**: Documents with shared high-IDF keywords exhibit strong Cosine similarity, while unrelated documents produce exact zero dot products.

---

6. Production Python Scikit-Learn Code

```
 from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [
    "database connection failed",
    "database query timeout",
    "billing payment declined"
]

vectorizer = TfidfVectorizer(smooth_idf=True, norm='l2')
tfidf_matrix = vectorizer.fit_transform(corpus)
sim_matrix = cosine_similarity(tfidf_matrix)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
print("Cosine Similarity Matrix:\n", sim_matrix.round(4))
```

---

7. Interview Questions & Production Trade-offs

What problem does TF-IDF solve over Bag-of-Words?

Bag-of-Words counts raw term frequencies, causing generic stop words (`"the"` , `"and"`) to dominate feature weights. TF-IDF multiplies TF by IDF to penalize words appearing across many documents.

Why was IDF introduced?

IDF measures word informativeness: rare domain keywords receive higher weights than ubiquitous structural words.

What are its limitations?

- **Destroys Word Order**: `"not good"` and `"good not"` yield identical vectors.
- **Polysemy & Synonymy**: Treats `"database"` and `"DB"` as unrelated dimensions (0.0 similarity).
- **High Dimensionality & Sparsity**: Vocabularies with $100,000$ terms create massive sparse matrices where $> 99\%$ of values are zeros.

Computational Complexity:

- **Training Indexing Complexity**: $O(|D| \times L)$ where L is average document token length.
- **Inference Query Complexity**: $O(k \times |V|)$ for sparse dot product.

Production Use Cases:

- BM25 Hybrid Keyword Search in RAG pipelines (combining sparse TF-IDF and dense vector embeddings).
- Fast baseline text classification for support ticket routing.

Follow-up Interview Questions:

1. *What is the difference between Sublinear TF and Linear TF?* (Answer: Sublinear TF uses $1 + \log(\text{TF})$ to prevent a word appearing 10 times from having 10x the weight of a word appearing once).
2. *Why is Cosine Similarity preferred over Euclidean Distance for TF-IDF vectors?* (Answer: Euclidean distance grows with document length, whereas Cosine similarity measures angle regardless of vector magnitude).

Module 04: Word Embeddings (Word2Vec, GloVe, FastText & Vector Space Geometry)

This study guide covers static dense embeddings, Word2Vec architectures (CBOW vs. Skip-Gram), a step-by-step Skip-Gram intuition walkthrough, FastText subword n-grams, GloVe co-occurrence intuition, t-SNE vector geometry plots, Gensim code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `04_word_embeddings_word2vec_glove.ipynb`

---

1. Limitations of Sparse Representations

Sparse models (One-Hot, TF-IDF) suffer from two core limitations:
1. **Orthogonality & Zero Similarity**: $\text{CosineSim}(\text{"database"}, \text{"postgres"}) = 0.0$ despite high semantic equivalence.
2. **Curse of Dimensionality**: Vector length equals vocabulary size ($|V| \geq 100,000$).

Dense embeddings project words into a continuous lower-dimensional space $\mathbb{R}^d$ ($d \approx 100 - 300$), capturing distributional semantics ("*words that occur in similar contexts tend to have similar meanings*").

---

2. Word2Vec Architectures: CBOW vs. Skip-Gram

Dimension	Continuous Bag-of-Words (CBOW)	Skip-Gram
Task Objective	Predict target word w_t given context	Predict context words w_{t+k} given target w_t
Training Speed	Faster ($O(d)$ per window update)	Slower ($O(m \cdot d)$ per window update)
Infrequent Words	Averages context; weaker on rare words	Produces high-quality vectors for rare words
Best Use Case	Large corpora, fast baseline training	Smaller datasets, rich semantic queries

---

3. Word2Vec Skip-Gram Intuition & Training Step Walkthrough

1. Context Window Sliding:

For sentence `"microservice communicates via grpc protocol"` with center word $w_I = \text{"communicates"}$ and context window radius $m = 1$:
- Target Center Word (w_I): `"communicates"`
- Context Words (w_O): `"microservice"`, `"via"`

2. Input Embedding Lookup:

Each word in V has an **Input Vector Matrix** $W \in \mathbb{R}^{|V| \times d}$ and an **Output Vector Matrix** $W' \in \mathbb{R}^{|V| \times d}$.
Lookup extracts input vector $v_{w_I} = W[w_I, :] \in \mathbb{R}^d$.

3. Dot Product & Logit Computation:

The raw dot product measures vector alignment between center word v_{w_I} and context candidate v'_{w_O}:

$$z = {v'_{w_O}}^\top v_{w_I}$$

4. Sigmoid Probability Output:

Applying sigmoid yields prediction probability $\hat{y} \in (0, 1)$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + \exp(-z)}$$

5. Why Negative Sampling Helps (Intuition):

Full Softmax requires summing $\sum_{w=1}^{|V|} \exp({v'_w}^\top v_{w_I})$ over all $|V| \geq 100,000$ words ($O(|V| \cdot d)$ complexity).
Negative Sampling converts full Softmax into $K + 1$ binary logistic classifications:
- Predict `1` for the true positive context pair (w_I, w_O).
- Predict `0` for K ($K \approx 5 - 20$) randomly drawn noise words $(w_I, w_k) \sim P_n(w)$.
Complexity drops from $O(|V| \cdot d) \rightarrow O(K \cdot d)$, accelerating training by $1,000\text{x}$.

6. One Weight Update Intuition:

If prediction $\hat{y} = 0.70$ for positive pair, prediction error is $e = (1.0 - 0.70) = 0.30$.
- **Gradient Update**: Input vector v_{w_I} shifts in direction of output vector v'_{w_O}:

$$\Delta v_{w_I} = \eta \cdot (1 - \hat{y}) \cdot v'_{w_O}$$

Positive context words pull center vectors closer, while negative noise words push them apart.

---

4. Optional Topic: FastText (Subword Character N-Grams)

FastText represents each word as a bag of character n-grams bounded by `<` and `>`.
For word `"where"` with $n = 3$:
- Character n-grams: `<wh` , `whe` , `her` , `ere` , `re>`
- Word vector: Sum of character n-gram vectors:
$$v_{\text{"where"}} = v_{\text{}} + v_{\text{}}$$

> **Key Advantage**: Handles Out-Of-Vocabulary (OOV) words by summing vectors of constituent character n-grams.

---

5. Optional Topic: GloVe (Global Vectors Co-Occurrence Objective)

GloVe combines global matrix factorization with local context windowing. It operates directly on global co-occurrence matrix X_{ij}:

$$\mathcal{L}_{\text{GloVe}} = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Where $f(X_{ij}) = \left(\frac{X_{ij}}{x_{\max}} \right)^\alpha$ caps the weighting of highly frequent stop word co-occurrences.

---

6. Word2Vec 2D t-SNE Projection & Vector Analogies

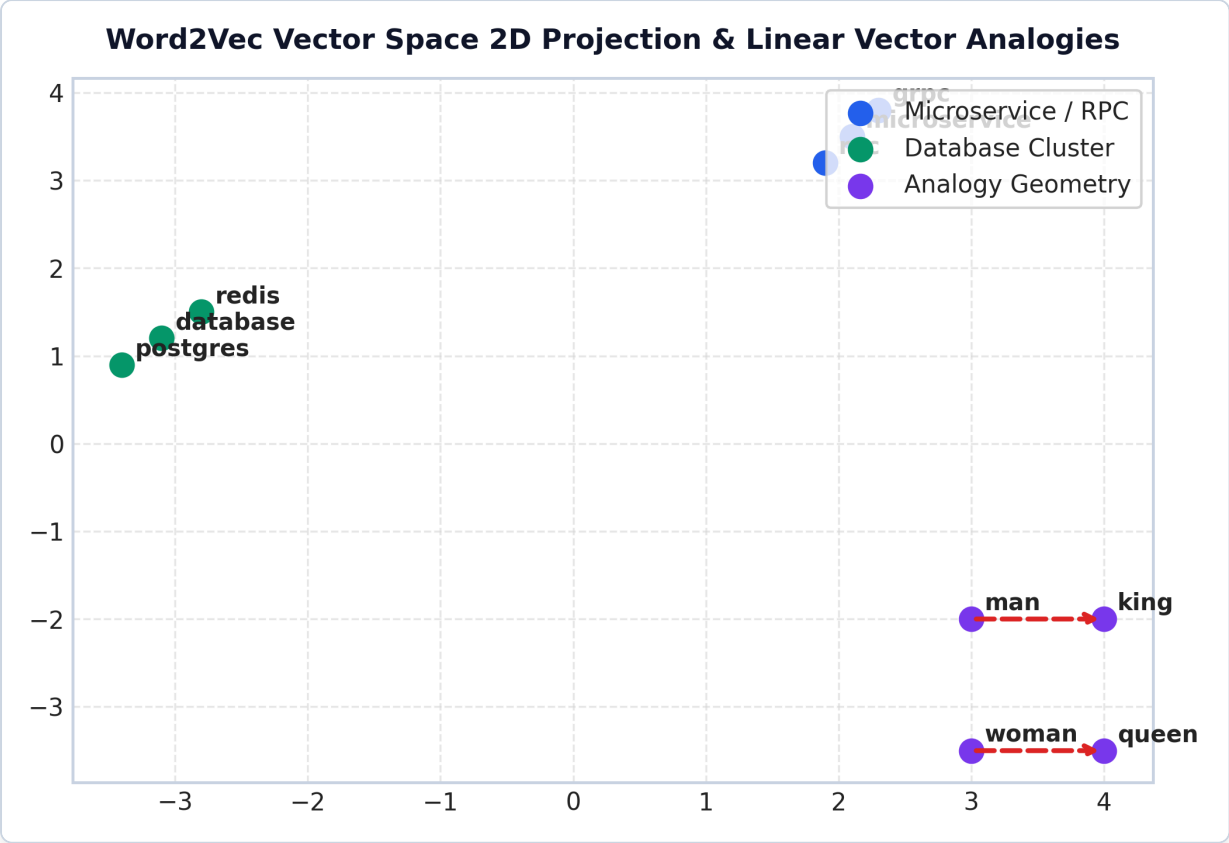

Figure: Word2Vec 2D t-SNE Vector Space

> **Plot Interpretation & Production Insight**:
> - **Dense Semantic Clustering**: Contextually similar words form tight clusters in vector space.
> - **Linear Vector Analogies**: Geometric offsets represent semantic relationships: $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$.

---

7. Production Gensim Word2Vec Python Code

```
from gensim.models import Word2Vec

corpus = [
    ["microservice", "communicates", "via", "grpc", "protocol"],
```

```
    ["database", "failover", "replica", "executed", "automatically"],
    ["grpc", "protocol", "ensures", "low", "latency", "rpc"],
    ["kubernetes", "deploys", "microservice", "container", "pods"]
]

model = Word2Vec(
    sentences=corpus,
    vector_size=50,
    window=2,
    min_count=1,
    sg=1,         # Skip-Gram
    negative=5,
    epochs=100
)

sim = model.wv.similarity("microservice", "grpc")
print("Similarity ('microservice', 'grpc'):", round(sim, 4))
```

---

8. Interview Questions & Production Trade-offs

What problem do dense word embeddings solve over sparse TF-IDF?

Dense embeddings project discrete tokens into continuous $\mathbb{R}^d$ space, resolving orthogonality and capturing semantic similarity ($\text{Sim} > 0.70$ for synonyms).

Why was Negative Sampling introduced?

Standard Skip-Gram Softmax requires computing partition function $\sum_{w=1}^{|V|} \exp(v'_w^\top v_{w_I})$ over all $|V|$ words. Negative Sampling approximates this with $K + 1$ binary logistic classifications, reducing time complexity from $O(|V|)$ to $O(K)$.

What are the primary limitations of Word2Vec?

- **Static Embeddings**: Assigns a single static vector per word, failing to handle polysemy (e.g. `"bank"` in river bank vs. investment bank gets identical vector).
- **Context Window Limit**: Only captures local co-occurrences within radius m, ignoring global document context.

Computational Complexity:

- **Training Time Complexity**: $O(C \cdot m \cdot K \cdot d)$ where C is corpus size, m is window radius, K is negative samples, and d is embedding dimension.
- **Inference Lookup Complexity**: $O(1)$ constant time vector array indexing.

Production Use Cases:

- Pre-trained embedding lookup table for classification architectures.
- Semantic query expansion in vector search engines.

Follow-up Interview Questions:

1. *How does FastText handle Out-Of-Vocabulary (OOV) terms during inference?* (Answer: It breaks the unseen OOV word into character n-grams, looks up their subword vectors, and averages them).
2. *What is the difference between Word2Vec and GloVe?* (Answer: Word2Vec trains online via local window SGD updates, whereas GloVe trains offline on global co-occurrence matrix log-counts).

Module 05: Classical NLP Models & Baseline Classifiers

This study guide covers Multinomial Naive Bayes, Laplace Add-1 smoothing, a step-by-step document classification numerical walkthrough, Logistic Regression, Linear SVM, Conditional Random Fields (CRF), ROC & Precision-Recall curves, Scikit-Learn pipeline code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `05_classical_nlp_models_and_baselines.ipynb`

---

1. Multinomial Naive Bayes & Laplace Add-1 Smoothing

Bayes' Theorem decomposes the posterior probability $P(c \mid d)$ of document $d = (w_1, w_2, \ldots, w_T)$ belonging to class c:

$$P(c \mid d) = \frac{P(c)P(d \mid c)}{P(d)}$$

Applying the **Naive Independence Assumption** (features are conditionally independent given class c):

$$P(c \mid d) \propto P(c) \prod_{i=1}^{T} P(w_i \mid c)$$

The Zero-Probability Problem & Laplace Add-1 Smoothing:

If a test document contains a word w_{new} that never appeared in training class c, raw maximum likelihood yields $P(w_{\text{new}} \mid c) = 0$, causing the entire posterior product to collapse to zero: $\prod P(w_i \mid c) = 0$.

Laplace Add-1 Smoothing adds a pseudo-count of 1 to every vocabulary word:

$$P(w_i \mid c) = \frac{N_{c,i} + 1}{N_c + |V|}$$

Where:
- $N_{c,i}$ is the count of word w_i in training class c.
- N_c is the total token count across all documents in class c.
- $|V|$ is the total unique vocabulary size.

---

2. Step-by-Step Document Classification Numerical Walkthrough

Suppose we have a binary classification task (**Spam** vs. **Ham**) with equal priors:
- $P(\text{Spam}) = 0.5$
- $P(\text{Ham}) = 0.5$

Vocabulary & Training Token Counts ($|V| = 5$ Unique Words):

Vocabulary: $V = [\text{"password"}, \text{"reset"}, \text{"invoice"}, \text{"urgent"}, \text{"account"}]$

- **Spam Training Token Counts**:
- `password`: 4, `reset`: 3, `urgent`: 3 ($N_{\text{Spam}} = 4 + 3 + 3 = 10$ total tokens)
- **Ham Training Token Counts**:
- `invoice`: 5, `account`: 3, `reset`: 2 ($N_{\text{Ham}} = 5 + 3 + 2 = 10$ total tokens)

Test Document: `"password reset"` (2 tokens)

Step 1: Compute Laplace Smoothed Likelihoods ($|V| = 5$)

- **For Class Spam** ($N_{\text{Spam}} = 10$, $N_{\text{Spam}} + |V| = 15$):
- $P(\text{"password"} \mid \text{Spam}) = \frac{4+1}{10+5} = \frac{5}{15} \approx 0.3333$
- $P(\text{"reset"} \mid \text{Spam}) = \frac{3+1}{10+5} = \frac{4}{15} \approx 0.2667$
- **For Class Ham** ($N_{\text{Ham}} = 10$, $N_{\text{Ham}} + |V| = 15$):
- $P(\text{"password"} \mid \text{Ham}) = \frac{0+1}{10+5} = \frac{1}{15} \approx 0.0667$ *(Notice: Laplace smoothing prevents zero probability!)*
- $P(\text{"reset"} \mid \text{Ham}) = \frac{2+1}{10+5} = \frac{3}{15} \approx 0.2000$

Step 2: Compute Unnormalized Posterior Probabilities

- $\text{Score}(\text{Spam}) = P(\text{Spam}) \times P(\text{"password"} \mid \text{Spam}) \times P(\text{"reset"} \mid \text{Spam})$

$$\text{Score}(\text{Spam}) = 0.5 \times \frac{5}{15} \times \frac{4}{15} = \frac{20}{450} \approx \mathbf{0.0444}$$

- $\text{Score}(\text{Ham}) = P(\text{Ham}) \times P(\text{"password"} \mid \text{Ham}) \times P(\text{"reset"} \mid \text{Ham})$

$$\text{Score}(\text{Ham}) = 0.5 \times \frac{1}{15} \times \frac{3}{15} = \frac{3}{450} \approx \mathbf{0.0067}$$

Step 3: Argmax Decision & Normalized Posterior Probability

- **Argmax Decision**: $\text{Score}(\text{Spam}) = 0.0444 > \text{Score}(\text{Ham}) = 0.0067 \implies \text{Spam}$
- **Normalized Confidence**:

$$P(\text{Spam} \mid \text{"password reset"}) = \frac{20}{20 + 3} = \frac{20}{23} \approx \mathbf{86.96\%}$$

---

3. Linear Classifiers: Logistic Regression & Linear SVM

For high-dimensional sparse TF-IDF vectors, linear baselines offer strong execution performance:
- **Logistic Regression**: Minimizes Log Loss $\mathcal{L}_{\text{CE}} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$, returning calibrated posterior probabilities.
- **Linear SVM (LinearSVC)**: Minimizes Hinge Loss $\mathcal{L}_{\text{Hinge}} = \max(0, 1 - y \cdot (w^\top x + b))$, maximizing boundary margin.

---

4. Classifier ROC & Precision-Recall Curves

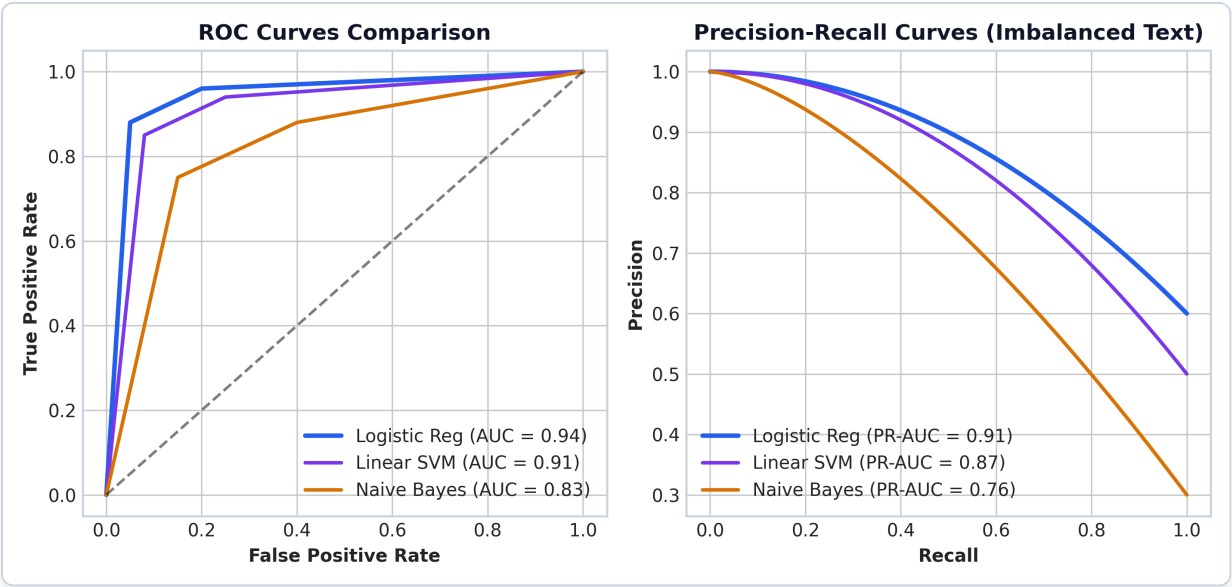

Figure: Classifier ROC & PR Curve Comparison

> **Plot Interpretation & Production Insight**:
> - **Imbalanced Text Datasets**: On imbalanced corpora (e.g. 95% normal logs, 5% security incidents), Precision-Recall curves provide a more realistic assessment than ROC curves.

---

5. Production Python Scikit-Learn Pipeline Code

```
 from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

X_train = [
    "database connection timeout on port 5432 postgresql replica",
    "billing credit card payment authorization declined error",
    "SQL query execution failed due to lock contention",
    "monthly invoice statement shows unexpected overage charge"
]
y_train = ["Infrastructure", "Billing", "Infrastructure", "Billing"]

pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("clf", LogisticRegression(C=1.0))
])

pipeline.fit(X_train, y_train)

test_ticket = ["postgresql database primary server timeout failure"]
pred = pipeline.predict(test_ticket)[0]
proba = max(pipeline.predict_proba(test_ticket)[0])

print(f"Predicted Category: {pred} (Confidence: {proba*100:.1f}%)")
```

---

6. Interview Questions & Production Trade-offs

What problem does Naive Bayes solve?

Provides an ultra-fast probabilistic baseline for text classification with $O(N)$ linear training time and minimal memory requirements.

Why is Laplace Smoothing required?

Without smoothing, any unseen vocabulary term in a class yields a zero probability likelihood $P(w_{\text{new}} \mid c) = 0$, causing the entire posterior product to collapse to zero regardless of other strong evidence.

What is the primary limitation of Naive Bayes?

The **Conditional Independence Assumption**: Naive Bayes assumes word occurrences are completely independent given the class label. In reality, text features are heavily correlated (e.g., `"credit"` and `"card"` co-occur together).

Computational Complexity:

- **Training Time Complexity**: $O(|D| \times L)$ single-pass vocabulary accumulation.
- **Inference Time Complexity**: $O(C \times L)$ per text query where C is class count and L is token length.

Production Use Cases:

- Real-time spam filtering and email routing engines.
- Initial baseline model in enterprise classification benchmark suites.

Follow-up Interview Questions:

1. *Why do we compute Naive Bayes using Log-Probabilities $\sum \log P(w_i \mid c)$ in code?* (Answer: Multiplying hundreds of small floating point probabilities leads to numerical underflow. Summing log-probabilities maintains numerical stability).
2. *When should you prefer Logistic Regression over Naive Bayes?* (Answer: When text features have strong correlations or when calibrated probability scores are required for confidence thresholding).

Module 06: Sequence Models (RNN, LSTM, GRU & Seq2Seq Architectures)

This study guide covers Recurrent Neural Networks (RNN), Backpropagation Through Time (BPTT), vanishing gradients, the 5 core LSTM update equations, a scalar LSTM forward pass walkthrough (forward pass only), GRU gating, PyTorch code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `06_sequence_models_rnn_lstm_gru.ipynb`

---

1. Vanilla RNN Architecture & Vanishing Gradients

Vanilla Recurrent Neural Networks process sequential input tokens $x_1, x_2, \dots, x_T$ step-by-step, maintaining a hidden state $h_t \in \mathbb{R}^d$:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

The Vanishing Gradient Problem in BPTT:

Calculating the loss gradient $\frac{\partial \mathcal{L}_T}{\partial h_k}$ at time step $k \ll T$ requires repeatedly multiplying weight matrix $W_{hh}^\top$:

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=k+1}^{T} W_{hh}^\top \text{diag}(1 - \tanh^2(\dots))$$

If the largest eigenvalue $\lambda_{\max}(W_{hh}) < 1.0$, the gradient norm decays exponentially to zero ($0.85^T \to 0$), preventing long-range dependency learning.

---

2. The 5 Core LSTM Update Equations

The Long Short-Term Memory (LSTM) architecture resolves vanishing gradients by introducing a cell memory state C_t governed by 3 gating mechanisms:

Where $\sigma(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid function and $\odot$ represents element-wise Hadamard product.

---

3. Step-by-Step Scalar LSTM Forward Pass Walkthrough

> 📌 **NOTE:**
> **Interview Scope**: Forward Pass Only (No BPTT derivation required).

Consider a single-cell LSTM forward pass step with simple scalar inputs:
- Inputs: $x_t = 1.0$, previous hidden state $h_{t-1} = 0.5$, previous cell state $C_{t-1} = 0.8$.
- Scalar Weights & Biases (set to $W = 1.0, b = 0.0$ for step demonstration):

Step 1: Compute Gating Signals

- Summed activation input: $z = h_{t-1} + x_t = 0.5 + 1.0 = 1.5$
- **Forget Gate**:

$$f_t = \sigma(1.5) = \frac{1}{1 + e^{-1.5}} \approx \mathbf{0.8176}$$

- **Input Gate**:

$$i_t = \sigma(1.5) \approx \mathbf{0.8176}$$

- **Candidate Cell State**:

$$\tilde{C}_t = \tanh(1.5) = \frac{e^{1.5} - e^{-1.5}}{e^{1.5} + e^{-1.5}} \approx \mathbf{0.9051}$$

Step 2: Cell State Update (C_t)

$$C_t = (f_t \times C_{t-1}) + (i_t \times \tilde{C}_t)$$

$$C_t = (0.8176 \times 0.8) + (0.8176 \times 0.9051) = 0.6541 + 0.7400 = \mathbf{1.3941}$$

Step 3: Output Gate & Hidden State (h_t)

- **Output Gate**:

$$o_t = \sigma(1.5) \approx \mathbf{0.8176}$$

- **Hidden State Output**:

$$h_t = o_t \times \tanh(C_t) = 0.8176 \times \tanh(1.3941)$$

$$\tanh(1.3941) \approx 0.8840$$

$$h_t = 0.8176 \times 0.8840 = \mathbf{0.7228}$$

---

4. Gated Recurrent Unit (GRU) Simplification

GRU merges cell state and hidden state into a single state h_t, utilizing only 2 gates:
1. **Reset Gate (r_t)**: Controls how much past hidden state to forget.
2. **Update Gate (z_t)**: Controls the balance between previous state h_{t-1} and candidate state $\tilde{h}_t$.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

GRU has 25% fewer parameters than LSTM, accelerating training on smaller datasets.

---

5. RNN vs. LSTM Gradient Flow Comparison

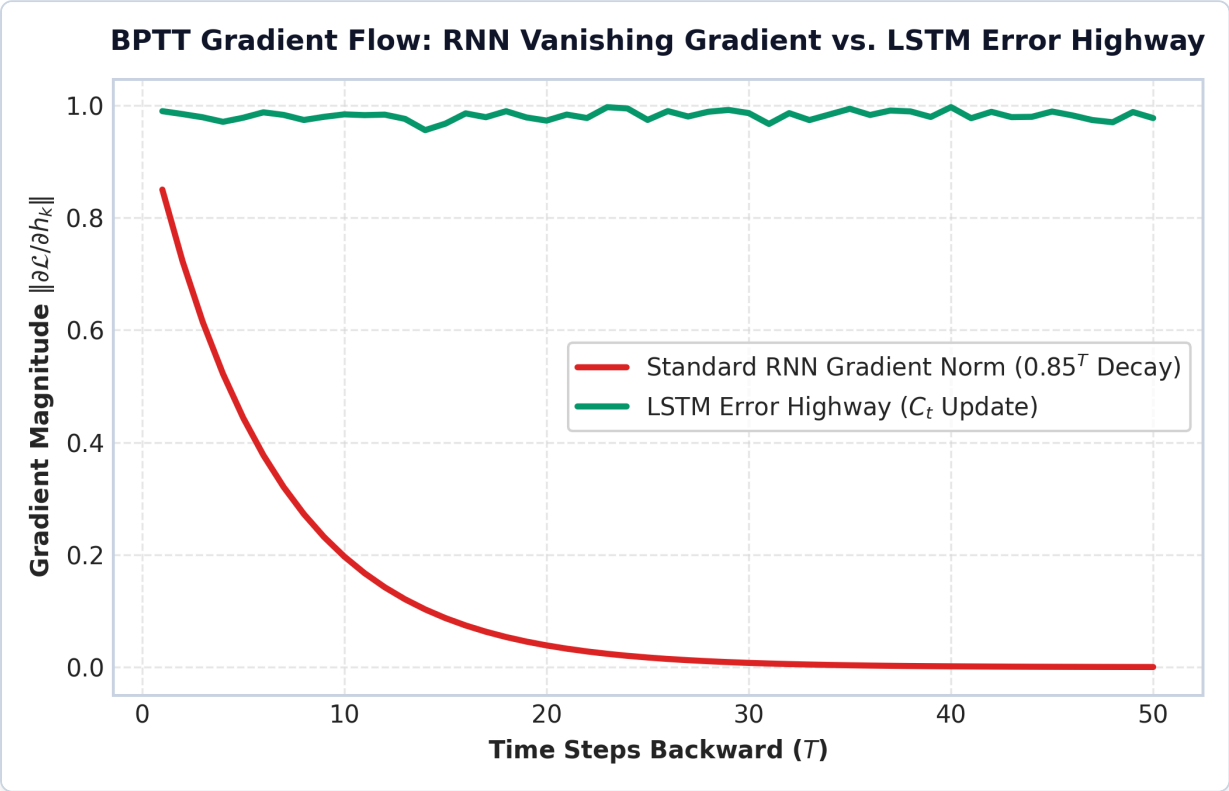

Figure: RNN Vanishing Gradient vs LSTM Error Highway

> **Plot Interpretation & Production Insight**:
> - **Error Highway**: The LSTM cell state C_t update is purely additive ($C_t = f_t \odot C_{t-1} + \dots$), creating a linear gradient highway that maintains gradient norm ≈ 1.0 across 50+ time steps.

---

6. Production PyTorch LSTM Implementation Code

```
 import torch
import torch.nn as nn

class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size=1000, embed_dim=64, hidden_dim=128, num_classes=2):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x):
        # x shape: (batch_size, seq_len)
        embedded = self.embedding(x)                    # (batch_size, seq_len, embed_dim)
        output, (hn, cn) = self.lstm(embedded)          # output: (batch_size, seq_len,
hidden_dim*2)
        # Concatenate forward and backward final hidden states
        final_hidden = torch.cat((hn[-2], hn[-1]), dim=1) # (batch_size, hidden_dim*2)
        return self.fc(final_hidden)                    # (batch_size, num_classes)

model = SentimentLSTM()
dummy_input = torch.randint(0, 1000, (4, 32))  # Batch size 4, seq len 32
output = model(dummy_input)
print("PyTorch LSTM Output Tensor Shape:", output.shape)
```

---

7. Interview Questions & Production Trade-offs

What problem do LSTMs solve over vanilla RNNs?

Vanilla RNNs suffer from vanishing gradients because backpropagating through time involves repeated matrix multiplication by $W_{hh}^{\top}$, causing gradients to decay exponentially to zero. LSTMs introduce an additive cell memory highway (C_t) that preserves gradient flow across long sequences ($T > 100$).

Why was the Cell State constant error highway introduced?

In cell update equation $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, if forget gate $f_t = 1$, the gradient derivative $\frac{\partial C_t}{\partial C_{t-1}} = 1$, allowing gradients to pass backward without matrix multiplication attenuation.

What are the primary limitations of LSTMs?

- **Sequential Bottleneck**: Hidden state h_t depends sequentially on h_{t-1}, preventing parallel GPU execution over sequence length T.
- **Fixed Hidden Capacity**: Forcing long sequences into fixed vector h_t creates an information bottleneck (resolved by Attention).

Computational Complexity:

- **Training Step Time Complexity**: $O(T \cdot (4 \cdot d^2 + 4 \cdot d \cdot d_x))$ per sequence.
- **Inference Time Complexity**: $O(T \cdot d^2)$ strictly sequential operations.

Production Use Cases:

- Time-series forecasting and sensor telemetry anomaly detection.
- Legacy speech recognition and optical character recognition (OCR) systems.

Follow-up Interview Questions:

1. *Why does bidirectional LSTM double the hidden dimension size?* (Answer: BiLSTM concatenates the forward hidden state $\vec{h}_t$ and backward hidden state $\overleftarrow{h}_t$, doubling output channels).
2. *When would you choose GRU over LSTM?* (Answer: When training on small datasets with limited GPU memory; GRU has fewer parameters and trains faster while matching LSTM accuracy).

Module 07: Attention Mechanisms (Bahdanau, Luong & Scaled Dot-Product Attention)

This study guide covers the Seq2Seq bottleneck, Bahdanau vs. Luong attention, Scaled Dot-Product Attention, an explicit mathematical breakdown of why $\sqrt{d_k}$ scaling is required, a 3-token numerical walkthrough, alignment heatmaps, PyTorch code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `07_attention_mechanisms_bahdanau_luong.ipynb`

---

1. The Information Bottleneck in Seq2Seq Models

In standard Encoder-Decoder RNNs, the encoder compresses an arbitrary input sequence $X_{1:T}$ into a single static hidden vector h_T. For long input sequences ($T > 30$), forcing all semantic context into a fixed-length vector creates an information bottleneck, leading to translation degradation.

Attention resolves this bottleneck by allowing the decoder to dynamically query and weight all encoder hidden states $h_1, \dots, h_T$ at every decoding step.

---

2. Bahdanau (Additive) vs. Luong (Multiplicative) Attention

Dimension	Bahdanau Attention (Additive)	Luong Attention (Multiplicative)
Score Function	$e_{ij} = v_a^\top \tanh(W_a s_{i-1} + W_b h_j)$	$e_{ij} = s_i^\top W_a h_j$
Decoder State	Uses previous decoder state s_{i-1}	Uses current decoder state s_i
Computation	Slower (requires matrix addition & tanh)	Faster (matrix multiplication optimized for GPUs)
Alignment Vector	Concatenated before RNN step	Combined after RNN hidden state computation

---

3. Scaled Dot-Product Attention Formulation

Modern Transformer architectures utilize **Scaled Dot-Product Attention** over Query (Q), Key (K), and Value (V) matrices:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Where:
- $Q \in \mathbb{R}^{T_q \times d_k}$ is the Query matrix.
- $K \in \mathbb{R}^{T_k \times d_k}$ is the Key matrix.

- $V \in \mathbb{R}^{T_k \times d_v}$ is the Value matrix.
- d_k is the Key vector projection dimension.

---

4. Why Scaling by $\sqrt{d_k}$ is Required (Crucial Interview Topic)

> **Interview Core Question**: *Why do we divide $QK^\top$ by $\sqrt{d_k}$ in Attention?*

Mathematical Proof & Intuition:

Suppose Query components q_i and Key components k_i are independent random variables with mean 0 and variance 1:

$$\mathbb{E}[q_i] = 0, \quad \mathrm{Var}(q_i) = 1, \quad \mathbb{E}[k_i] = 0, \quad \mathrm{Var}(k_i) = 1$$

Their dot product is the sum of d_k products: $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$.
1. **Expected Value**: $\mathbb{E}[q \cdot k] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = 0$
2. **Variance**: Since variables are independent, variances add up:

$$\mathrm{Var}(q \cdot k) = \sum_{i=1}^{d_k} \mathrm{Var}(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k$$

The Softmax Saturation Problem:

As dimension d_k grows large (e.g. $d_k = 64$ or 512), the magnitude of dot products $|q \cdot k|$ grows to $\sqrt{d_k} \approx 8 - 22$.

When inputs to the Softmax function become extremely large, Softmax outputs saturate into extremely sharp peak distributions (≈ 1.0 for the max, ≈ 0.0 for all others). In these saturated regions, Softmax gradients become virtually zero ($\mathrm{softmax}'(z) \to 0$), causing **vanishing gradients during backpropagation**.

Dividing by $\sqrt{d_k}$ scales variance back to 1.0:

$$\mathrm{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{\mathrm{Var}(q \cdot k)}{d_k} = \frac{d_k}{d_k} = 1.0$$

This keeps Softmax inputs in a well-behaved range where gradients flow cleanly.

---

5. Step-by-Step 3-Token Numerical Walkthrough

Consider a 3-token sequence ($T = 3$) with projection dimension $d_k = 2 \implies \sqrt{d_k} = \sqrt{2} \approx 1.4142$.

Given Input Matrices ($Q, K \in \mathbb{R}^{3\times 2}, V \in \mathbb{R}^{3\times 2}$):

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 1 \\ 0 & 2 \\ 1 & 0 \end{bmatrix}$$

Step 1: Compute Raw Dot Product Matrix $QK^\top$

$$QK^\top = \begin{bmatrix}1 & 0\\0 & 1\\1 & 1\end{bmatrix}\begin{bmatrix}1 & 1 & 0\\0 & 1 & 1\end{bmatrix} = \begin{bmatrix}(1\cdot1+0) & (1\cdot1+0) & (0+0)\\(0+0) & (0+1) & (0+1)\\(1\cdot1+0) & (1\cdot1+1\cdot1) & (0+1\cdot1)\end{bmatrix} = \begin{bmatrix}1 & 1 & 0\\0 & 1 & 1\\1 & 2 & 1\end{bmatrix}$$

Step 2: Scale by $\frac{1}{\sqrt{2}} \approx 0.7071$

$$S = \frac{QK^\top}{\sqrt{2}} = \begin{bmatrix}0.7071 & 0.7071 & 0.0000\\0.0000 & 0.7071 & 0.7071\\0.7071 & 1.4142 & 0.7071\end{bmatrix}$$

Step 3: Compute Row-wise Softmax Probabilities ($\alpha = \text{softmax}(S)$)

- **Row 1**: Exponents: $[e^{0.7071}, e^{0.7071}, e^{0}] = [2.0281, 2.0281, 1.0000]$, Sum $= 5.0562$

$$\alpha_1 = \left[\frac{2.0281}{5.0562}, \frac{2.0281}{5.0562}, \frac{1.0000}{5.0562}\right] = [\mathbf{0.4011}, \mathbf{0.4011}, \mathbf{0.1978}]$$

- **Row 2**: Exponents: $[e^{0}, e^{0.7071}, e^{0.7071}] = [1.0000, 2.0281, 2.0281]$, Sum $= 5.0562$

$$\alpha_2 = [\mathbf{0.1978}, \mathbf{0.4011}, \mathbf{0.4011}]$$

- **Row 3**: Exponents: $[e^{0.7071}, e^{1.4142}, e^{0.7071}] = [2.0281, 4.1132, 2.0281]$, Sum $= 8.1694$

$$\alpha_3 = \left[\frac{2.0281}{8.1694}, \frac{4.1132}{8.1694}, \frac{2.0281}{8.1694}\right] = [\mathbf{0.2483}, \mathbf{0.5035}, \mathbf{0.2483}]$$

$$\alpha = \begin{bmatrix}0.4011 & 0.4011 & 0.1978\\0.1978 & 0.4011 & 0.4011\\0.2483 & 0.5035 & 0.2483\end{bmatrix}$$

Step 4: Multiply by Value Matrix V ($O = \alpha V$)

- **Row 1 Output**:

$$O_1 = 0.4011\begin{bmatrix}2 & 1\end{bmatrix} + 0.4011\begin{bmatrix}0 & 2\end{bmatrix} + 0.1978\begin{bmatrix}1 & 0\end{bmatrix}$$

$$O_1 = \begin{bmatrix}(0.8022 + 0 + 0.1978), & (0.4011 + 0.8022 + 0)\end{bmatrix} = \begin{bmatrix}1.0000 & 1.2033\end{bmatrix}$$

---

6. Luong Attention Alignment Heatmap

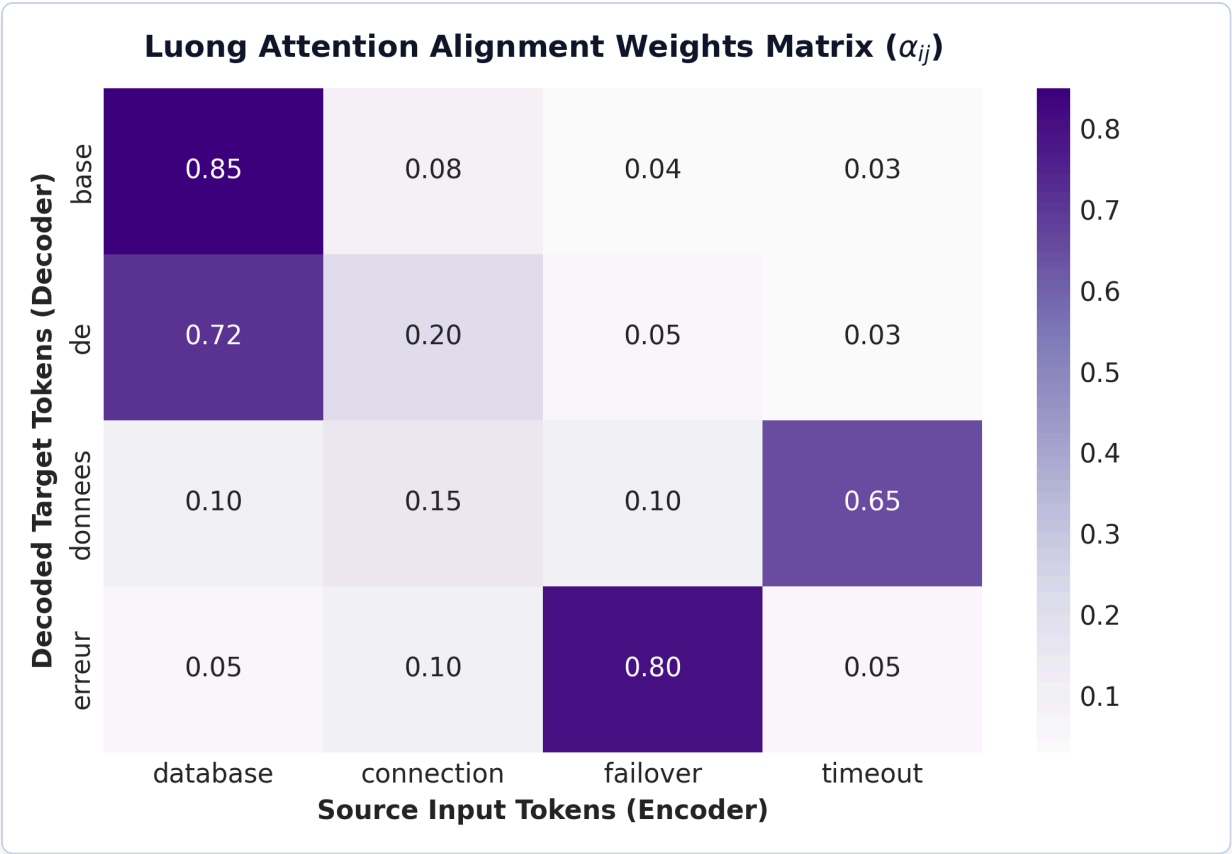

Figure: Luong Attention Alignment Heatmap

> **Plot Interpretation & Production Insight**:
> - **Alignment Weights**: High attention values along diagonal tokens show dynamic focus allocation across tokens.

---

7. Production PyTorch Scaled Dot-Product Attention Code

```python
import torch
import torch.nn.functional as F

# Query, Key, Value Tensors: Batch size 1, 3 Tokens, Dimension 2
Q = torch.tensor([[[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]]])
K = torch.tensor([[[1.0, 0.0], [1.0, 1.0], [0.0, 1.0]]])
V = torch.tensor([[[2.0, 1.0], [0.0, 2.0], [1.0, 0.0]]])

# Execute PyTorch optimized Scaled Dot-Product Attention
output = F.scaled_dot_product_attention(Q, K, V)

print("=== PyTorch Scaled Dot-Product Attention Output ===")
print("Output Matrix (Shape: 1x3x2):\n", output.numpy().round(4))
```

---

8. Interview Questions & Production Trade-offs

What problem does Attention solve over standard Encoder-Decoder RNNs?

Standard Encoder-Decoder RNNs force all source tokens into a single static hidden vector h_T, creating a severe information bottleneck. Attention allows the decoder to dynamically inspect and weight all source token hidden states at every step.

Why is scaling by $\sqrt{d_k}$ required?

Without $\sqrt{d_k}$ scaling, large projection dimensions d_k cause dot products $q \cdot k$ to have high variance d_k. Large dot products push the Softmax function into saturated regions with near-zero gradients ($\text{softmax}'(z) \to 0$), causing vanishing gradients during backpropagation.

What are the primary limitations of Self-Attention?

Quadratic Time & Memory Complexity $O(T^2)$ over sequence length T, making vanilla self-attention expensive for long contexts ($T > 8,000$).

Computational Complexity:

- **Time Complexity**: $O(T^2 \cdot d)$ matrix multiplication over sequence length T.
- **Memory Complexity**: $O(T^2)$ for storing $T \times T$ attention matrix weights in RAM.

Production Use Cases:

- Core building block of Transformer LLMs (GPT-4, Llama 3, Claude 3).
- Cross-attention in RAG contextual retrieval decoders.

Follow-up Interview Questions:

1. *What is the difference between Self-Attention and Cross-Attention?* (Answer: In Self-Attention, Q, K, V stem from the same sequence. In Cross-Attention, Q comes from the decoder, while K, V come from the encoder).
2. *How does Multi-Head Attention improve representation capacity?* (Answer: It splits d into h heads, allowing the model to attend to different representation subspaces simultaneously, such as syntactic relations in head 1 and positional offsets in head 2).

Module 08: NLP Evaluation Metrics (BLEU, ROUGE & Perplexity)

This study guide covers BLEU (modified precision & brevity penalty), ROUGE-1 and ROUGE-L, Perplexity ($\text{PPL} = \exp(\mathcal{L})$), step-by-step numerical walkthroughs, metric plots, NLTK/Rouge-Score Python code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `08_nlp_evaluation_metrics_bleu_rouge.ipynb`

---

1. Overview of Generation Evaluation Metrics

Evaluating machine translation, summarization, and LLM text generation requires automated metrics that compare candidate generation c against reference text r:

Metric	Primary Orientation	Core Objective Equation	Key Focus Area
BLEU	Precision-Oriented	$\text{BP} \cdot \exp\left(\sum w_n \log p_n\right)$	Machine Translation fidelity & adequacy
ROUGE-1	Recall-Oriented	$\frac{\text{Matching Unigrams}}{\text{Ref Unigram Count}}$	Text Summarization reference coverage
ROUGE-L	Sequence Order Recall	$\frac{\text{LCS}(c,r)}{\text{Ref Token Length } r}$	Longest Common Subsequence sentence structure
Perplexity	Model Uncertainty	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$	Language Model next-token prediction

---

2. BLEU Score (Bilingual Evaluation Understudy)

Why BLEU is Precision-Oriented:

Standard raw precision counts how many candidate words appear in the reference. If a flawed model outputs repeating high-confidence words (`"the the the the"`), raw precision yields 100% $(4/4)$.

BLEU introduces **Modified N-Gram Precision (p_n)**, which clips each candidate n-gram count to the maximum frequency it appears in any reference sentence.

Brevity Penalty (BP):

To prevent candidate sentences from cheating by outputting extremely short high-precision phrases (e.g. `"the cat"`), BLEU multiplies precision by a **Brevity Penalty**:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{if } c \leq r \end{cases}$$

Where c is candidate word length and r is reference word length.

BLEU-2 Formula:

$$\text{BLEU-2} = \text{BP} \cdot \exp\left(0.5 \log p_1 + 0.5 \log p_2\right)$$

Common Limitations of BLEU:

- **Surface-Form Exact Match Rigidity**: Penalizes valid synonyms, morphological variations, and paraphrases (e.g. `"quick"` vs `"fast"` gets zero match).
- **Insensitive to Word Order Shifts**: Does not measure long-range semantic coherence.

---

3. Step-by-Step BLEU-2 Numerical Walkthrough

- **Candidate ($c = 5$ words)**: `"the cat sat on mat"`
- **Reference ($r = 6$ words)**: `"the cat sat on the mat"`

Step 1: Compute Modified Unigram Precision (p_1)

- Candidate Unigrams ($c = 5$): `["the", "cat", "sat", "on", "mat"]`
- Reference Unigram Counts: `the`: 2, `cat`: 1, `sat`: 1, `on`: 1, `mat`: 1
- All 5 candidate unigrams match reference counts $\implies$ $\text{Clipped Count} = 5$.

$$p_1 = \frac{5}{5} = \mathbf{1.0000}$$

Step 2: Compute Modified Bigram Precision (p_2)

- Candidate Bigrams ($c - 1 = 4$): `["the cat", "cat sat", "sat on", "on mat"]`
- Reference Bigrams: `["the cat", "cat sat", "sat on", "on the", "the mat"]`
- Matching Bigrams: `"the cat"`, `"cat sat"`, `"sat on"` (3 matches).

$$p_2 = \frac{3}{4} = \mathbf{0.7500}$$

Step 3: Compute Brevity Penalty (BP)

Since candidate length $c = 5 < r = 6$:

$$\text{BP} = \exp\left(1 - \frac{6}{5}\right) = \exp(-0.20) \approx \mathbf{0.8187}$$

Step 4: Compute Final BLEU-2 Score

$$\text{BLEU-2} = 0.8187 \times \exp(0.5 \log 1.0 + 0.5 \log 0.7500)$$

$$\log 1.0 = 0, \quad \log 0.7500 \approx -0.2877$$

$$\text{BLEU-2} = 0.8187 \times \exp(-0.1438) = 0.8187 \times 0.8660 = \mathbf{0.7090}$$

---

4. ROUGE Metrics (ROUGE-1 & ROUGE-L)

ROUGE-1 (Unigram Recall):

$$\text{ROUGE-1 Recall} = \frac{\text{Matching Unigrams}}{\text{Total Reference Unigrams}}$$

ROUGE-L (Longest Common Subsequence):

$$\text{ROUGE-L Recall} = \frac{\text{LCS}(c, r)}{r}$$

Where $\text{LCS}(c, r)$ is the length of the longest common subsequence of tokens between candidate and reference.

Step-by-Step ROUGE Numerical Walkthrough:

- Candidate: `"the cat sat on mat"` (5 words)
- Reference: `"the cat sat on the mat"` (6 words)

1. **ROUGE-1 Recall**:
   Matching unigrams = 5, Total reference unigrams = 6.

$$\text{ROUGE-1 Recall} = \frac{5}{6} \approx \mathbf{0.8333}$$

2. **ROUGE-L Recall**:
   Longest Common Subsequence: `"the" -> "cat" -> "sat" -> "on" -> "mat"` (length = 5).

$$\text{ROUGE-L Recall} = \frac{5}{6} \approx \mathbf{0.8333}$$

---

5. Perplexity (PPL)

Perplexity measures a language model's uncertainty when predicting the next token in a sequence $W = (w_1, w_2, \ldots, w_N)$:

$$\text{PPL}(W) = \exp(\mathcal{L}_{\text{CrossEntropy}}) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_{

> **Core Interpretation**: **Lower Perplexity** $\implies$ **Better Language Model**. A perplexity of $\text{PPL} = 10.0$ means the model is as uncertain at each token choice as if it were selecting uniformly at random from 10 candidate words.

Step-by-Step PPL Calculation:

If an evaluation run yields Cross-Entropy Loss $\mathcal{L}_{\text{CE}} = 2.3026$:

$$\text{PPL} = \exp(2.3026) = \mathbf{10.0}$$

---

6. Perplexity vs. Cross-Entropy Loss Curve

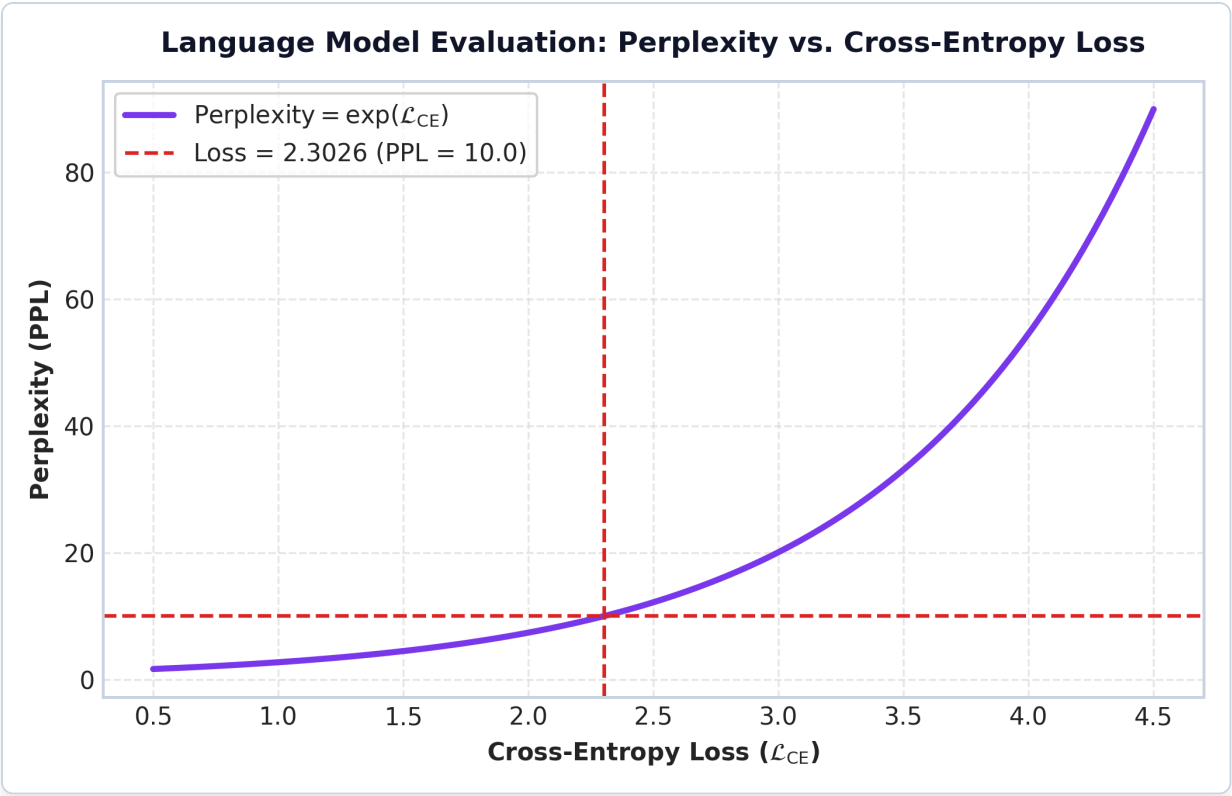

Figure: Perplexity vs Loss Curve

> **Plot Interpretation & Production Insight**:
> - Exponential mapping: As evaluation loss decreases linearly, perplexity drops exponentially, providing a clear benchmark metric during LLM pre-training and fine-tuning.

---

7. Production Python NLTK & ROUGE Evaluation Code

```
 from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from rouge_score import rouge_scorer

candidate = "the cat sat on mat"
reference = "the cat sat on the mat"

# BLEU Evaluation
weights = (0.5, 0.5)  # BLEU-2
bleu_2 = sentence_bleu([reference.split()], candidate.split(), weights=weights)

# ROUGE Evaluation
scorer = rouge_scorer.RougeScorer(['rouge1', 'rougeL'], use_stemmer=True)
rouge_scores = scorer.score(reference, candidate)

print(f"BLEU-2 Score: {bleu_2:.4f}")
print(f"ROUGE-1 Recall: {rouge_scores['rouge1'].recall:.4f}")
print(f"ROUGE-L Recall: {rouge_scores['rougeL'].recall:.4f}")
```

---

8. Interview Questions & Production Trade-offs

What problem do BLEU, ROUGE, and Perplexity solve?

Provide automated, reproducible quantitative benchmarks for text generation without relying on expensive human annotations.

Why is BLEU precision-oriented while ROUGE is recall-oriented?

BLEU checks what fraction of generated candidate n-grams are valid (preventing hallucinated ungrounded text in translation), whereas ROUGE checks what fraction of reference target information was captured in the summary (preventing key detail omission).

What are their primary limitations?

Surface-form exact match rigidity. They penalize valid paraphrases and modern conversational outputs. In GenAI production pipelines, **LLM-as-a-Judge** (using GPT-4 / G-Eval to evaluate response quality) is heavily preferred over BLEU/ROUGE.

Computational Complexity:

- **BLEU / ROUGE Time Complexity**: $O(|c| \cdot |r|)$ sequence token matching.
- **Perplexity Time Complexity**: $O(N)$ forward pass evaluation loss computation.

Production Use Cases:

- Automated regression testing during LLM fine-tuning runs (e.g. LoRA / QLoRA checkpoints).
- Translation engine quality assurance benchmarking.

Follow-up Interview Questions:

1. *Why does lower perplexity guarantee better model probability estimation but NOT guaranteed fluent text generation?* (Answer: Perplexity measures next-token probability fit on validation data, but greedy or temperature decoding strategies can still suffer from repetitive loops during generation).
2. *What is G-Eval / LLM-as-a-Judge?* (Answer: A evaluation framework where an advanced LLM evaluates generated responses against rubrics like factual accuracy, relevance, and toxicity).

Module 09: Enterprise Applications & Production Serving Pipelines

This study guide covers real-time vs. batch serving architectures, INT8 quantization, ONNX Runtime acceleration, production benchmark plots, PyTorch inference code, complexity analysis, and standardized interview Q&A.

> **Notebook Companion**: `09_nlp_applications_and_end_to_end_pipelines.ipynb`

---

1. Real-Time Streaming vs. Batch Serving Architectures

Dimension	Real-Time API Serving (gRPC / REST)	Batch Inference (Spark / Ray / Kafka)
Latency Target	Strict SLA ($p95 < 20\text{ms}$)	High Throughput (Hours / Nightly runs)
Batch Size	Small ($B = 1 - 8$) to minimize latency	Large ($B = 128 - 1024$) to maximize GPU utilization
Hardware	CPU instances or quantized edge GPUs	High-memory GPU clusters (A100 / H100)
Optimization Focus	ONNX Runtime, TensorRT, INT8 Quantization	Distributed data-parallel workers

---

2. Production Optimization Techniques

1. INT8 Quantization:

Converts 32-bit floating-point weights (W_{FP32}) to 8-bit integers (W_{INT8}):

$$W_{\text{INT8}} = \text{round}\left(\frac{W_{\text{FP32}}}{S}\right) + Z$$

Where S is scale factor and Z is zero-point offset.
- **RAM Footprint Reduction**: Reduces memory footprint by 75% ($400\text{MB} \rightarrow 100\text{MB}$).
- **Throughput Acceleration**: Utilizes vector SIMD AVX-512 / VNNI CPU instructions.

2. ONNX Runtime Acceleration:

Open Neural Network Exchange (ONNX) fuses adjacent linear layers and matrix operations into optimized execution graphs, eliminating Python interpreter overhead.

---

3. Production Serving Benchmark Comparison

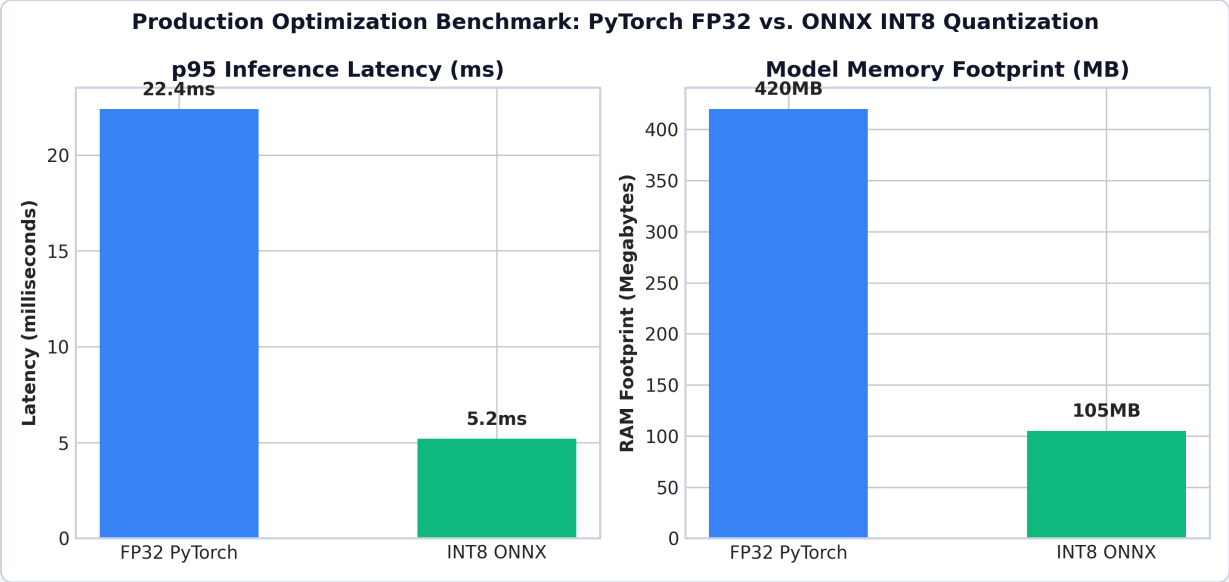

Figure: FP32 PyTorch vs INT8 ONNX Latency & Memory Benchmark

Plot Interpretation & Production Insight:
- **Latency Drop**: ONNX INT8 reduces $p95$ latency from 22.4ms down to 5.2ms (4.3x throughput boost).
- **RAM Savings**: Reduces model footprint from 420MB to 105MB, enabling 4x more worker replicas per server node.

---

4. Production Python PyTorch & ONNX Pipeline Code

```
 import torch
import torch.nn as nn
import time

class ProductionClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(100, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 2)

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

model = ProductionClassifier()
model.eval()

# Quantize Model to INT8 Dynamic
quantized_model = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
)

dummy_input = torch.randn(1, 100)

start = time.perf_counter()
```

```
output = quantized_model(dummy_input)
latency_ms = (time.perf_counter() - start) * 1000

print(f"INT8 Model Output Shape: {output.shape}")
print(f"Single Request Latency: {latency_ms:.3f} ms")
```

---

5. Interview Questions & Production Trade-offs

What problem does INT8 Quantization solve?

FP32 neural network models require high RAM and CPU/GPU memory bandwidth, resulting in slow inference latencies ($> 20\text{ms}$) and high cloud infrastructure costs. INT8 quantization compresses weight matrices by 75% (4x memory reduction) while utilizing INT8 SIMD vector instructions for faster execution.

Why was ONNX Runtime introduced?

Native PyTorch/TensorFlow models carry Python interpreter overhead and un-fused execution graphs. ONNX exports models to a framework-agnostic computation graph, enabling graph optimizations (layer fusion, constant folding) across heterogeneous hardware backends (CPU, CUDA, TensorRT).

What are the primary trade-offs of quantization?

Slight loss in numerical precision ($< 0.5\%$ accuracy drop). Highly sensitive layers (like attention softmax or final output logits) may require Mixed Precision (FP16) or selective quantization.

Computational Complexity:

- **INT8 Inference Time Complexity**: $O(N \cdot d)$ with 4x SIMD instruction throughput.
- **Memory Footprint Complexity**: $O(\frac{1}{4}M_{\text{FP32}})$ bytes.

Production Use Cases:

- High-throughput web microservices serving real-time predictions.
- Mobile and edge device deployment (iOS CoreML, Android NNAPI).

Follow-up Interview Questions:

1. *What is the difference between Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT)?* (Answer: PTQ quantizes weights after training without re-training, while QAT models quantization error during training forward passes to preserve accuracy on sensitive tasks).
2. *How does Dynamic Quantization differ from Static Quantization?* (Answer: Dynamic quantization converts weights to INT8 offline but keeps activations in FP32 until runtime, whereas Static quantization quantizes both weights and activations to INT8 using calibration datasets).

Module 10: Classical NLP vs. Modern LLMs Master Cheatsheet & 30 Flashcards

This master study guide provides a single-page architectural comparison matrix, a time & memory complexity matrix, and 30 high-frequency interview flashcards for Data Science, AI, Applied AI, GenAI, and ML Engineer interviews.

---

1. Master Architectural Comparison Matrix

Dimension	TF-IDF + Logistic	Word2Vec / GloVe	LSTM / GRU	Transformer / LLMs
Representation	Sparse $\mathbb{R}^{\|V\|}$	Dense Static $\mathbb{R}^d$	Contextual $\mathbb{R}^{T \times d}$	Deep Contextual $\mathbb{R}^{T \times d}$
Polysemy Handling	None (1 vector per word)	None (1 vector per word)	Dynamic per sequence step	Dynamic Multi-Head Self-Attention
Context Horizon	Bag-of-Words / Local	Local Window ($m = 2-5$)	Sequential ($T \approx 100$)	Global Context ($128k+$ tokens)
Parallel Training	Fully Parallel (CPUs)	Fully Parallel (CPUs)	Sequential ($O(T)$ Bottleneck)	Fully Parallelized GPU Matrix Multiply
Inference Latency	Ultra-fast ($< 1\text{ms}$)	Lookup ($< 1\text{ms}$)	Fast ($5\text{ms} - 20\text{ms}$)	Autoregressive ($50\text{ms} - 1000\text{ms}$)
Primary Bottleneck	Feature Sparsity	Static Polysemy	Sequential Training	Quadratic Memory $O(T^2)$

---

2. Computational Complexity & Memory Footprint Matrix

Algorithm / Architecture	Training Time Complexity	Inference Time Complexity	Memory Footprint Complexity
TF-IDF Indexing	$O(\|D\| \times L)$	$O(k \times \|V\|)$	$O(\|D\| \times \|V\|)$ Sparse
Multinomial Naive Bayes	$O(\|D\| \times L)$	$O(C \times L)$	$O(C \times \|V\|)$ Dense
Word2Vec Skip-Gram	$O(C \cdot m \cdot K \cdot d)$	$O(1)$ Lookup	$O(\|V\| \times d)$ Embedding Table
LSTM Model Layer	$O(T \cdot d^2)$ per sequence	$O(T \cdot d^2)$ Sequential	$O(4 \cdot d^2)$ Model Weights
Scaled Dot-Product Attention	$O(T^2 \cdot d)$ per batch	$O(T^2 \cdot d)$	$O(T^2)$ RAM Attention Weights
INT8 Quantized Serving	$O(N \cdot d)$ SIMD	$O(N \cdot d)$ SIMD	$O(\frac{1}{4}M_{\text{FP32}})$ Bytes

---

3. 30 High-Frequency Interview Flashcards

1. Preprocessing & Tokenization

1. **Q: Why does Zipf's law necessitate subword tokenization?**
   - *A*: Zipf's law states $f(k) \propto 1/k$. Treating words as atomic units causes a long tail of rare words, inflating $|V|$ and causing high Out-Of-Vocabulary (OOV) rates. Subword tokenization (BPE) breaks rare words into known sub-units, achieving 0% OOV.
2. **Q: Compare Stemming and Lemmatization.**
   - *A*: Stemming chops suffixes using heuristic rules (fast, outputs invalid words like `"comput"`). Lemmatization uses morphological dictionaries and POS tags (slower, outputs valid canonical lemmas like `"compute"`).
3. **Q: What is the Byte Pair Encoding (BPE) algorithm?**
   - *A*: A bottom-up tokenization algorithm that initializes a vocabulary with individual characters and iteratively merges the most frequent adjacent symbol pair until reaching target vocabulary size $|V|$.
4. **Q: What is the Type-Token Ratio (TTR)?**
   - *A*: $\text{TTR} = \frac{|V|}{N_{\text{total}}}$. Measures lexical diversity in a corpus.

2. Traditional Text Representations & TF-IDF

5. **Q: What is the TF-IDF formula?**
   - *A*: $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$, where $\text{IDF}(t) = \log\left(\frac{1+|D|}{1+\text{DF}(t)}\right) + 1$.
6. **Q: Why is IDF introduced?**
   - *A*: IDF downweights ubiquitous stop words (`"the"`, `"is"`) while magnifying rare, domain-specific keywords (`"postgresql"`, `"billing"`).
7. **Q: Why is L2 normalization applied to TF-IDF vectors?**
   - *A*: To eliminate document length bias so long documents do not produce artificially larger vector norms.
8. **Q: Why is Cosine Similarity preferred over Euclidean Distance for TF-IDF?**
   - *A*: Cosine similarity measures vector orientation angle rather than magnitude, preventing document length from distorting similarity.

3. Word Embeddings (Word2Vec, GloVe, FastText)

9. **Q: Compare Word2Vec CBOW and Skip-Gram.**
   - *A*: CBOW predicts target word w_t from surrounding context (faster). Skip-Gram predicts context words from target word w_t (better representation for rare words).
10. **Q: Why was Negative Sampling introduced in Word2Vec?**
    - *A*: Full Softmax requires computing partition function $\sum_{w=1}^{|V|} \exp(v'_w^\top v_{w_I})$ over all $|V|$ words ($O(|V|)$ complexity). Negative Sampling converts this into $K+1$ binary logistic classifications ($O(K)$ complexity).
11. **Q: What is the primary limitation of static embeddings like Word2Vec?**
    - *A*: Static embeddings assign a single fixed vector per word, failing to resolve polysemy (e.g. `"bank"` in river bank vs. investment bank).
12. **Q: How does FastText handle Out-Of-Vocabulary (OOV) terms?**

- *A*: FastText represents words as bags of character n-grams. Unseen OOV words are embedded by summing the vectors of their constituent character n-grams.

13. **Q: How does GloVe differ from Word2Vec?**
    - *A*: Word2Vec updates online via local context windows using SGD, while GloVe factorizes the global co-occurrence log-count matrix offline.

4. Classical Classifiers & Baselines

14. **Q: Write Bayes' Theorem with Naive Independence.**
    - *A*: $P(c \mid d) \propto P(c) \prod_{i=1}^{T} P(w_i \mid c)$. Assumes features are conditionally independent given class c.

15. **Q: Why is Laplace Add-1 Smoothing necessary in Naive Bayes?**
    - *A*: Without smoothing, an unseen word in class c yields $P(w_{\text{new}} \mid c) = 0$, causing the entire posterior product to collapse to zero. Laplace smoothing sets $P(w_i \mid c) = \frac{N_{c,i}+1}{N_c+|V|}$.

16. **Q: Why log-probabilities are used when implementing Naive Bayes?**
    - *A*: Summing log-probabilities $\sum \log P(w_i \mid c)$ prevents floating-point numerical underflow caused by multiplying hundreds of small probabilities.

17. **Q: Why are Precision-Recall curves preferred over ROC curves for imbalanced text?**
    - *A*: On imbalanced datasets (e.g. 95% normal, 5% anomaly), ROC curves can appear overly optimistic because False Positive Rate stays small. Precision-Recall curves isolate minority positive class performance.

5. Sequence Models (RNN, LSTM, GRU)

18. **Q: What causes vanishing gradients in Vanilla RNNs during BPTT?**
    - *A*: Backpropagating over T steps requires multiplying weight matrix $W_{hh}^{\top}$ repeatedly. If $\lambda_{\max}(W_{hh}) < 1.0$, gradients decay exponentially ($0.85^T \to 0$).

19. **Q: What are the 5 core LSTM update equations?**
    - *A*: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$, $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$, $\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$, $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \implies h_t = o_t \odot \tanh(C_t)$.

20. **Q: How does the LSTM Cell State (C_t) eliminate vanishing gradients?**
    - *A*: The cell state update $C_t = f_t \odot C_{t-1} + \dots$ acts as an additive linear highway. If $f_t = 1$, the gradient derivative $\frac{\partial C_t}{\partial C_{t-1}} = 1$, preserving gradient flow across time steps.

21. **Q: Compare LSTM and GRU.**
    - *A*: GRU merges cell state and hidden state, using 2 gates (Reset r_t and Update z_t). GRU has 25% fewer parameters and trains faster.

22. **Q: What is the main architectural bottleneck of RNN/LSTM models?**
    - *A*: Sequential computation dependency (h_t depends on h_{t-1}), preventing parallel GPU training over sequence length T.

6. Attention Mechanisms

23. **Q: What is the Scaled Dot-Product Attention formula?**
    - *A*: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$.

24. **Q: Why do we scale by $\sqrt{d_k}$ in Attention?**

- *A*: For large d_k, dot products $q \cdot k$ have variance d_k. Large dot products push Softmax into saturated regions with near-zero gradients ($\text{softmax}'(z) \to 0$), causing vanishing gradients during backpropagation. Dividing by $\sqrt{d_k}$ rescales variance back to 1.0.

25. **Q: Compare Bahdanau and Luong Attention.**
    - *A*: Bahdanau uses additive score function $v_a^\top \tanh(W_a s_{i-1} + W_b h_j)$. Luong uses multiplicative score function $s_i^\top W_a h_j$ (optimized for GPU matrix multiplication).

26. **Q: What is the computational complexity of Self-Attention?**
    - *A*: $O(T^2 \cdot d)$ time and $O(T^2)$ RAM memory complexity over sequence length T.

7. Evaluation Metrics & Serving

27. **Q: What is BLEU score and why is it precision-oriented?**
    - *A*: BLEU measures modified n-gram precision multiplied by a Brevity Penalty. It focuses on precision to prevent models from outputting repetitive high-confidence tokens.

28. **Q: What is ROUGE and how does ROUGE-1 differ from ROUGE-L?**
    - *A*: ROUGE is recall-oriented for summarization. ROUGE-1 measures unigram recall, while ROUGE-L measures Longest Common Subsequence (LCS) sentence structure.

29. **Q: What is Perplexity and how is it related to Cross-Entropy Loss?**
    - *A*: $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$. Lower perplexity indicates a better language model with lower next-token branching uncertainty.

30. **Q: What is INT8 Quantization?**
    - *A*: Converts 32-bit floating point weights (W_{FP32}) to 8-bit integers (W_{INT8}), achieving 75% RAM savings (4x memory compression) and accelerating CPU inference via SIMD instructions.